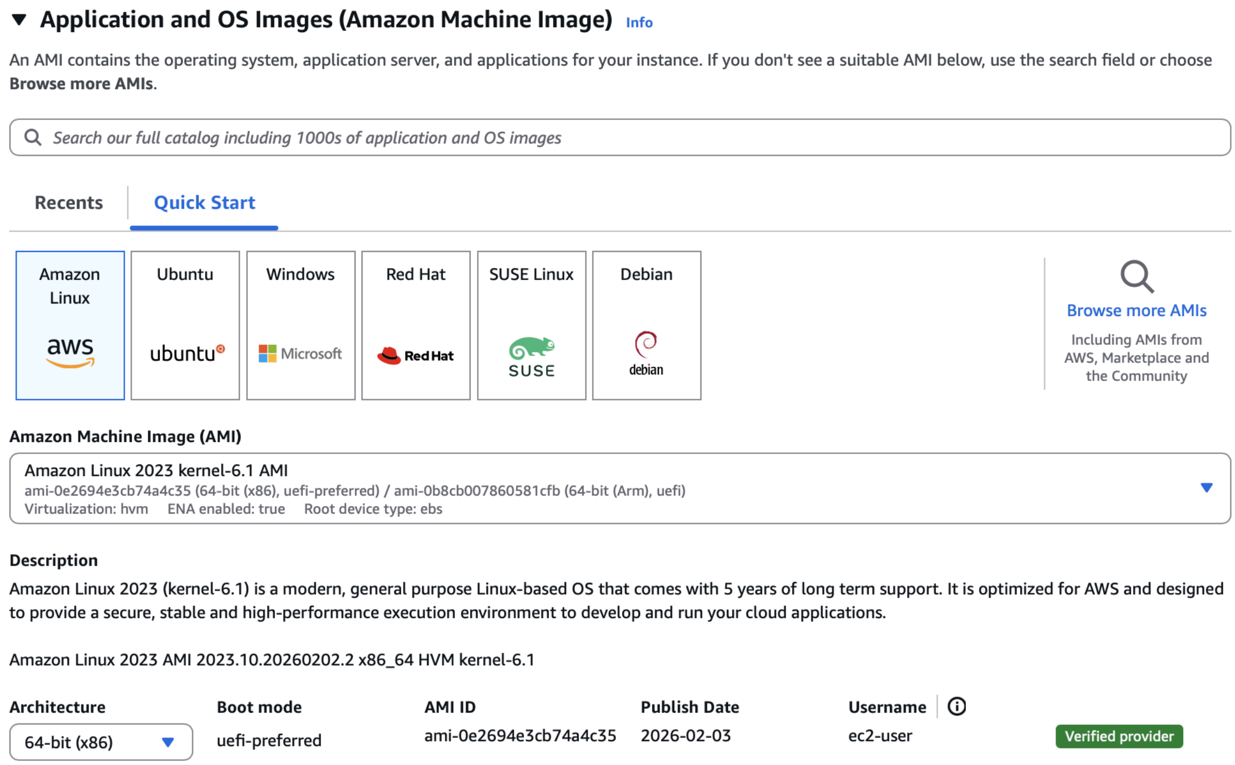
1. Setup Terraform

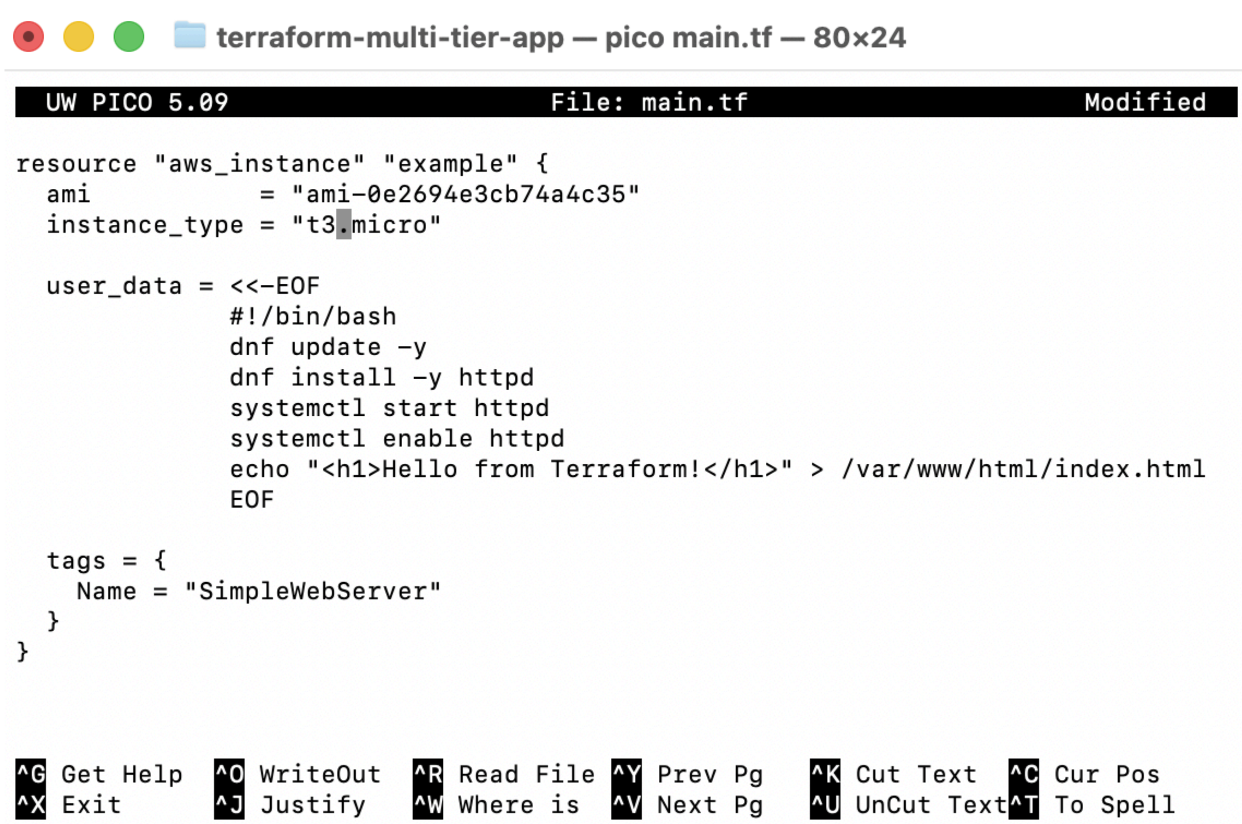
```
jane@Janes-MacBook-Air terraform-multi-tier-app % brew upgrade hashicorp/tap/terraform
terraform -v

==> Upgrading 1 outdated package:
hashicorp/tap/terraform 1.5.7 -> 1.14.5
==> Fetching downloads for: terraform
✔ Formula terraform (1.14.5)                         Verified     28.7MB/ 28.7MB
==> Upgrading hashicorp/tap/terraform
  1.5.7 -> 1.14.5
🍺  /opt/homebrew/Cellar/terraform/1.14.5: 5 files, 95.6MB, built in 2 seconds
==> Running `brew cleanup terraform`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
Removing: /opt/homebrew/Cellar/terraform/1.5.7... (7 files, 65.3MB)
Removing: /Users/jane/Library/Caches/Homebrew/terraform_bottle_manifest--1.5.7-1... (8.5KB)
Removing: /Users/jane/Library/Caches/Homebrew/terraform--1.5.7... (19.8MB)
==> `brew cleanup` has not been run in the last 30 days, running now...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
Removing: /Users/jane/Library/Caches/Homebrew/portable-ruby-3.4.8.arm64_big_sur.bottle.tar.gz... (12.2MB)
Removing: /Users/jane/Library/Caches/Homebrew/api-source/Homebrew/homebrew-cask/d23244137d2e620b04bd2e3ef32c7582b012e441/Cask/docker-desktop.rb... (5.1KB)
Removing: /Users/jane/Library/Caches/Homebrew/bootsnap/50726f7c717931c314b0629398fd1e28fb0b942293f91a9eac20addacdfb7399... (651 files, 5.6MB)
Removing: /Users/jane/Library/Logs/Homebrew/postgresql@14... (1.3KB)
Removing: /Users/jane/Library/Logs/Homebrew/glib... (64B)
Removing: /Users/jane/Library/Logs/Homebrew/llvm... (64B)
Removing: /Users/jane/Library/Logs/Homebrew/gcc... (64B)
Removing: /Users/jane/Library/Logs/Homebrew/fontconfig... (73KB)
Removing: /Users/jane/Library/Logs/Homebrew/openssl@3... (64B)
Removing: /Users/jane/Library/Logs/Homebrew/ca-certificates... (64B)
Removing: /Users/jane/Library/Logs/Homebrew/node... (64B)
Pruned 0 symbolic links and 2 directories from /opt/homebrew
Terraform v1.14.5
on darwin_arm64
```

2. Define AWS Provider

```
jane@Janes-MacBook-Air terraform-multi-tier-app % nano provider.tf
jane@Janes-MacBook-Air terraform-multi-tier-app % ls
cat provider.tf

provider.tf
provider "aws" {
  region = "us-west-1"
}

jane@Janes-MacBook-Air terraform-multi-tier-app % terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.32.1...
- Installed hashicorp/aws v6.32.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

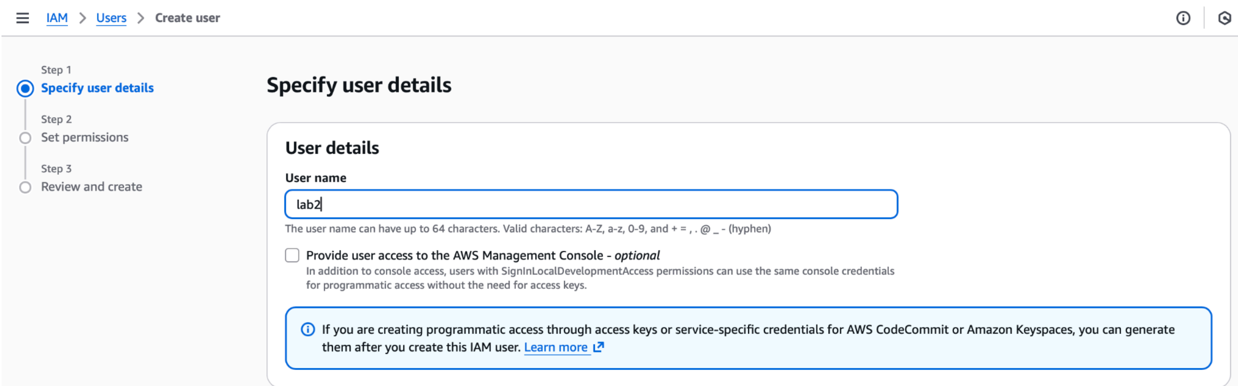
3. Create an EC2 Instance






```
resource "aws_instance" "example" {
  ami           = "ami-0e2694e3cb74a4c35"
  instance_type = "t3.micro"

  user_data = <<-EOF
              #!/bin/bash
              dnf update -y
              dnf install -y httpd
              systemctl start httpd
              systemctl enable httpd
              echo "<h1>Hello from Terraform!</h1>" > /var/www/html/index.html
              EOF

  tags = {
    Name = "SimpleWebServer"
  }
}
```


4. Apply Configuration



IAM > Users > Create user

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Specify user details

User details

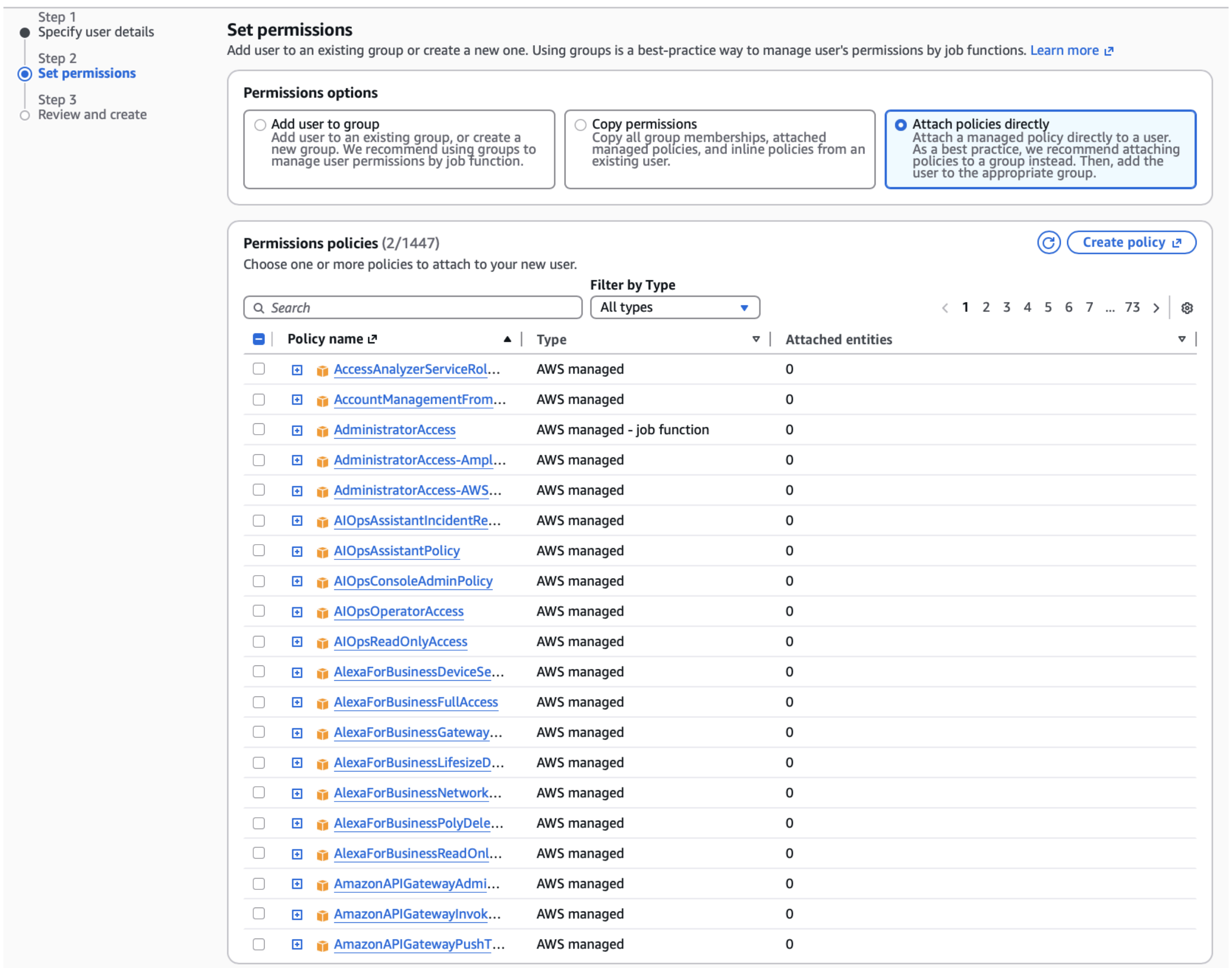
User name

lab2

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

Provide user access to the AWS Management Console - *optional*
In addition to console access, users with SignInLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. Learn more



Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. Learn more

Permissions options

- Add user to group
  Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- Copy permissions
  Copy all group memberships, attached managed policies, and inline policies from an existing user.
- Attach policies directly
  Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (2/1447)

Create policy

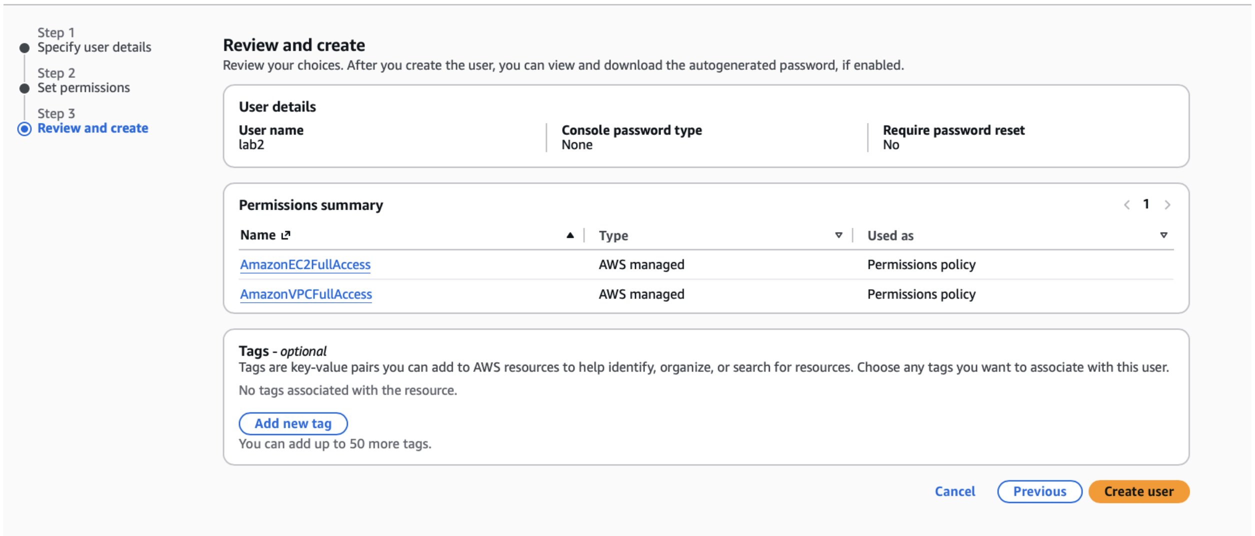
Choose one or more policies to attach to your new user.

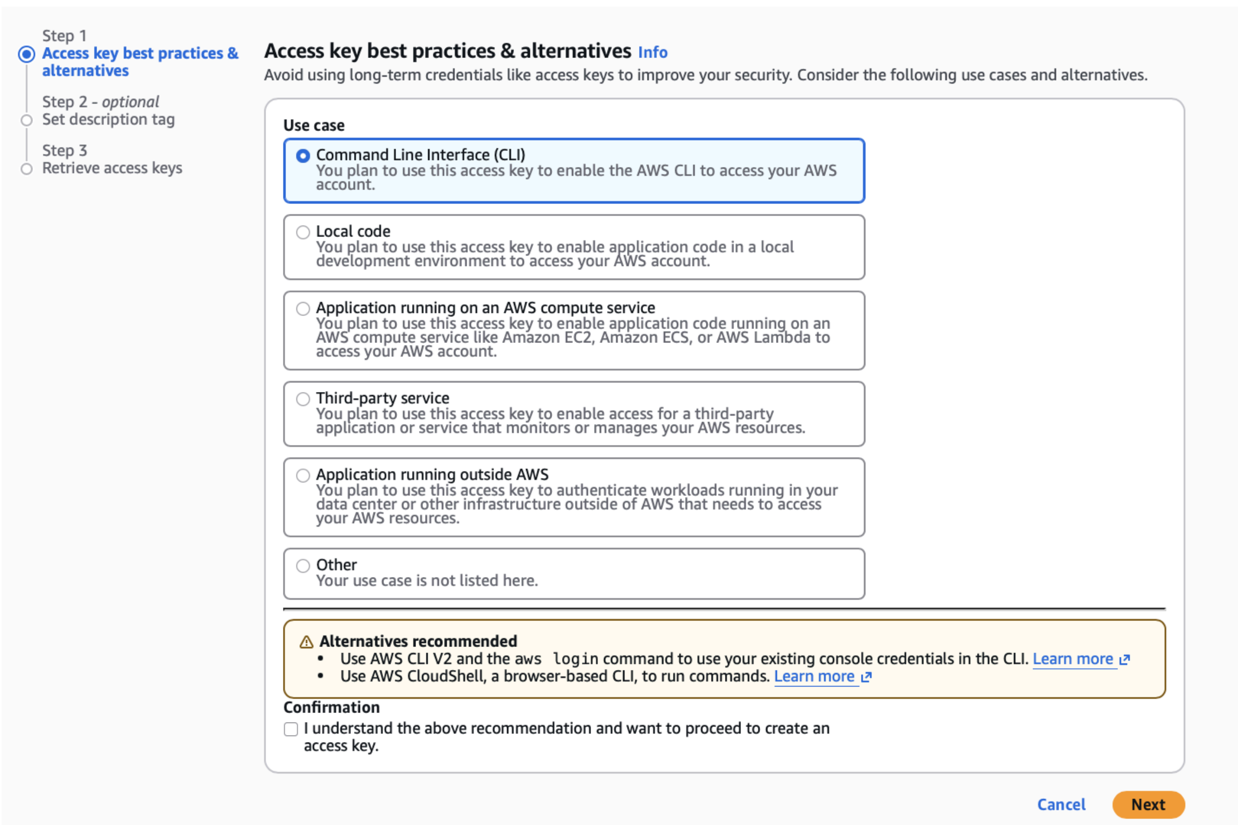
Filter by Type

Search | All types

< 1 2 3 4 5 6 7 ... 73 >

Policy name	Type	Attached entities
AccessAnalyzerServiceRol...	AWS managed	0
AccountManagementFrom...	AWS managed	0
AdministratorAccess	AWS managed - job function	0
AdministratorAccess-Ampl...	AWS managed	0
AdministratorAccess-AWS...	AWS managed	0
AIOpsAssistantIncidentRe...	AWS managed	0
AIOpsAssistantPolicy	AWS managed	0
AIOpsConsoleAdminPolicy	AWS managed	0
AIOpsOperatorAccess	AWS managed	0
AIOpsReadOnlyAccess	AWS managed	0
AlexaForBusinessDeviceSe...	AWS managed	0
AlexaForBusinessFullAccess	AWS managed	0
AlexaForBusinessGateway...	AWS managed	0
AlexaForBusinessLifesizeD...	AWS managed	0
AlexaForBusinessNetwork...	AWS managed	0
AlexaForBusinessPolyDele...	AWS managed	0
AlexaForBusinessReadOnl...	AWS managed	0
AmazonAPIGatewayAdmi...	AWS managed	0
AmazonAPIGatewayInvok...	AWS managed	0
AmazonAPIGatewayPushT...	AWS managed	0


Step 1
Specify user details
Step 2
Set permissions
Step 3
Review and create
Review and create
Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.
User details
User name
lab2
Console password type
None
Require password reset
No
Permissions summary
1
Name
Type
Used as
AmazonEC2FullAccess
AWS managed
Permissions policy
AmazonVPCFullAccess
AWS managed
Permissions policy
Tags - optional
Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.
No tags associated with the resource.
Add new tag
You can add up to 50 more tags.
Cancel
Previous
Create user


Step 1
Access key best practices & alternatives
Step 2 - optional
Set description tag
Step 3
Retrieve access keys
Access key best practices & alternatives Info
Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.
Use case
Command Line Interface (CLI)
You plan to use this access key to enable the AWS CLI to access your AWS account.
Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.
Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.
Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.
Application running outside AWS
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.
Other
Your use case is not listed here.
Alternatives recommended
Use AWS CLI V2 and the aws login command to use your existing console credentials in the CLI. Learn more
Use AWS CloudShell, a browser-based CLI, to run commands. Learn more
Confirmation
I understand the above recommendation and want to proceed to create an access key.
Cancel
Next

This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

Step 1
Access key best practices & alternatives

Step 2 - *optional*
Set description tag

Step 3
Retrieve access keys

Retrieve access keys Info

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key	Secret access key
AKIAS7EPZ35N5X5OU5OE	*************** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the best practices for managing AWS access keys.

Download .csv file | Done

lab2 Info

Delete

Summary

ARN arn:aws:iam::204312534875:user/lab2	Console access Disabled	Access key 1 AKIAS7EPZ35N5X5OU5OE - Active Never used. Created today.
Created February 13, 2026, 22:21 (UTC-08:00)	Last console sign-in -	Access key 2 Create access key

Permissions | Groups | Tags (1) | **Security credentials** | Last Accessed

Console sign-in

Enable console access

Console sign-in link
https://204312534875.signin.aws.amazon.com/console

Console password
Not enabled

Multi-factor authentication (MFA) (0)

Remove | Resync | Assign MFA device

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. Learn more

Type	Identifier	Certifications	Created on

No MFA devices. Assign an MFA device to improve the security of your AWS environment

Assign MFA device

Access keys (1)

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more

AKIAS7EPZ35N5X5OU5OE

Actions

Description
terraform-lab2

Status
Active

Last used
None

Created
Now

Last used region
N/A

Last used service
N/A

API keys for Amazon Bedrock (0)

Actions | Generate API Key

Use API keys for Amazon Bedrock to integrate into your library of choice and make API requests programmatically. You can have a maximum of two long-term API keys (active, inactive, or expired) at a time. Learn more

API key name	Created	Expires	Status

No Amazon Bedrock API keys

Generate API Key

```
jane@Janes-MacBook-Air terraform-multi-tier-app % terraform apply

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.example will be created
  + resource "aws_instance" "example" {
      + ami                                  = "ami-0e2694e3cb74a4c35"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "us-west-1"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Name" = "SimpleWebServer"
        }
      + tags_all                             = {
          + "Name" = "SimpleWebServer"
        }
      + tenancy                              = (known after apply)
      + user_data                            = <<-EOT
            #!/bin/bash
            dnf update -y
            dnf install -y httpd
            systemctl start httpd
            systemctl enable httpd
            echo "<h1>Hello from Terraform!</h1>" > /var/www/html/index.html
        EOT
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.example: Creating...
aws_instance.example: Still creating... [00m10s elapsed]
aws_instance.example: Creation complete after 13s [id=i-0b34f8044c2d94a18]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

5. Check Web Server